# Tutorial 8

## Exercise 1: Backtracking for knapsack

1. Apply the backtracking algorithm for Knapsack to the following instance: $W = 600$, $T = 14$, and

| $j$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| $w_j$ | 400 | 300 | 500 | 100 |
| $c_j$ | 2 | 5 | 10 | 5 |

2. How can the algorithm as presented during the lecture be improved?

---

## Exercise 2: The subset sum problem

In the *subset sum problem*, we are given a target $t \in \mathbb{N}$ and a sequence $n_1, n_2, \ldots, n_k$ of natural numbers, and the question is whether a subset of the $n_j$ sums up to exactly $t$. For example, given $1, 2, 3, 4, 5$ and $t = 10$ the answer is yes ($1 + 4 + 5 = 10$), given $1, 3, 3, 5, 7$ and $t = 2$ the answer is no (note that the 1 in the sequence can only be used once in the sum).

1. Formulate the problem as a decision problem (Hint: recall the definition of Knapsack).
2. Give a backtracking algorithm for the subset sum problem.
3. Apply your algorithm to the instance with $t = 307$ and the sequence $160, 56, 158, 123, 67, 26$.

---

## Exercise 3: DPLL

Apply the DPLL algorithm to the following formulas to determine whether they are satisfiable or unsatisfiable.

Illustrate the execution of the algorithm is we did in the lecture using a tree structure, i.e., showing the simplified formula for each recursive call and the rule applied. Assume that DPLL selects variables in alphabetical order (i.e., $A, B, C, D, E, \ldots$) if a rule is applicable to more than one variable (e.g., if there is more than one unit clause).

1. $\varphi_1 = (A \vee B) \wedge (B \vee C) \wedge (\neg B \vee D) \wedge (\neg A \vee \neg D) \wedge (\neg C \vee E) \wedge (A \vee \neg B \vee \neg D)$
2. $\varphi_2 = (\neg A \vee \neg B \vee \neg C) \wedge (A \vee \neg B) \wedge (A \vee \neg D) \wedge (B \vee \neg E) \wedge (\neg C \vee D) \wedge (C \vee E)$ $\wedge (C \vee \neg E) \wedge (\neg D \vee E)$

---

## Exercise 4: Sudoku

Describe how to encode a standard $9 \times 9$ Sudoku by an instance of SAT, i.e., given an incomplete Sudoku generate a formula so that a satisfying assignment to the formula encodes a valid completion of the Sudoku.

First, think about which variables to use, then about how to express the required constraints.